DidaTrade: Replicated Trading with Multi-Paxos, Fast Paxos, and Membership Changes

Group 13
João Agostinho Martim Afonso Samuel Sacher
Instituto Superior Técnico

Abstract

We present DidaTrade, a replicated trading service implemented in Java and gRPC, supporting Multi-Paxos, Fast Paxos, causally consistent balance reads, and dynamic membership. The implementation extends the provided baseline with pipelined consensus instances, Fast vote tracking and recovery, and state transfer across changing acceptor sets. This report focuses on the implementation-specific data structures and coordination mechanisms used to preserve consensus safety, ordered execution, and progress during configuration changes.

1 Introduction

DidaTrade implements all five project steps, extending the baseline with correct Paxos Phase 1 recovery, concurrent Multi-Paxos instances, Fast Paxos, causally ordered balance reads, and dynamic membership. Trading commands are ordered through a replicated consensus log and executed deterministically at each replica, while balance reads are served locally after the client's causal dependencies have been satisfied.

Our architecture separates incoming RPC handling from consensus coordination, per-instance Paxos state, request bookkeeping, peer communication, and ordered execution. This separation enables client requests to be registered independently of the consensus process, while dedicated components coordinate concurrent proposals, handle leader changes, and recover state during reconfiguration.

Section 2 describes the components of a server and their roles, and Section 3 explains how the different requests move through them.

2 Architecture

This section explains the structure of a server and the roles of its most important classes. How the different requests move through these components is described in Section 3.

The server has three main layers:

- **Services** (`service` package): receive the gRPC requests. The `DidaTradeMainServiceImpl` handles client requests, the `DidaTradePaxosServiceImpl` handles the acceptor and learner side of the Paxos requests from peers, and the `DidaTradeMasterServiceImpl` handles the console, which starts and activates ballots and sets debug modes.
- **Core** (`didatrade.server` package): processes the requests, runs consensus and executes decided commands.
- **Peer communication** (`PeerRpc`): sends all Paxos requests to the other servers.

2.1 Threading Model

Three kinds of threads work on a server:

- **gRPC threads** run the services. A client request thread blocks until its command has been executed.
- **One main loop thread** coordinates consensus and executes decided commands.
- **Pipeline workers** run the individual phase two proposals.

Shared state is synchronized on the `DidaTradeServerState`, while each `PaxosInstance` synchronizes itself.

2.2 Server State and Debug Modes

The `DidaTradeServerState` holds all relevant properties of a server, such as the `DebugState`, the `MainLoop`, the `RequestHistory`, the `PaxosLog`, the `TradeManager` and the current ballot.

The `DebugState` encapsulates the current debug mode. It exposes an `apply()` method that blocks, throttles or exits the server when called. Every incoming client request passes through it first, before anything else is done.

2.3 Request History

If the `DebugState` lets the server resume, the client command is written into the `RequestHistory`. It holds all requests and marks them as *pending* until they are executed, after which they are moved to *processed* together with their result.

2.4 Main Loop and Pipeline

The `MainLoop` is the single coordinator of the server. Every iteration, it

1. lets the executor execute all newly decided commands,
2. schedules consensus, and
3. sleeps until it is woken up, e.g. by a new client request or a finished proposal, or until the next retry is due.

Consensus is scheduled either when the server is the current leader or when a fast round is active. If the server thinks it has just become leader, the main loop first runs phase one.

It uses the `PaxosPipeline` to limit the number of concurrent Phase 2 proposals. Each slot maintains a local proposal claim, preventing multiple workers from proposing it simultaneously. The proposal state also stores a retry deadline and pins the Phase 2 value for each slot and ballot, ensuring that retries of the same slot cannot propose a different value under the same ballot.

2.5 Proposer

All proposal logic is encapsulated in the `PaxosProposer`. It runs both phase one and phase two proposals and uses the results to

- update the server state,
- store the responses in the `PaxosLog`, and
- notify other threads, such as the `MainLoop`, to move on with their work if necessary.

The responses of the peers are collected by response processors, one for each kind of request. One worth mentioning is the `PhaseOneResponseProcessor`, which uses an `AcceptedResponseObserver` per instance to track all votes for a value. It works for both classic and Fast Paxos.

2.6 Paxos Log and Paxos Instances

The `PaxosLog` does the high-level coordination of the log. It creates new Paxos instances, provides an interface to set proposed values into a log position, returns log entries, etc.

The core holder of information is the `PaxosInstance`, one per log slot. If a value is decided, the instance holds this value, its ballot, and whether the server has already executed it. Besides that, it is split into three nested classes:

- `LearnInfo` holds all learned values about the slot that the server received from its peers. It decides whether a quorum has been reached, i.e. whether the value is decided. If so, the instance adopts the value.
- `Accepted` holds what this server accepted as an acceptor in this slot. This is returned to a leader that asks for phase one.
- `LocalProposal` holds what this server tries to propose in this slot, and when a failed proposal may be retried.

2.7 Executor

The `OrderedPaxosExecutor` stores which slot it has to execute next. It executes the decided commands strictly in log order on the `TradeManager`, and stops at the first slot that is not decided yet, or whose command has not reached this server yet. After executing a command, it stores the result in the `RequestHistory`, which unblocks the waiting client request.

2.8 Fast Paxos and State Transfer

Fast Paxos and State Transfer each need an additional class. The `FastPaxosAcceptor` lets acceptors vote for client requests directly, without a leader proposal. `StateTransfer` installs a log prefix that the leader already recovered onto the new members.

A `membership_transferred` marker records whether the recovered prefix has been installed. The leader confirms the handoff only after a quorum of the new acceptor configuration acknowledges the transfer. This marker is also reported during Phase 1, allowing subsequent recovery to distinguish completed transfers from interrupted ones.

Apart from that, most of the components above also contain branching logic for both cases, which is explained further in Section 3.

3 Request Processing Pipeline

This section follows a client request through the components above. The request arrives in the same way for Classic and Fast Paxos, but the two paths assign slots and collect votes differently.

3.1 Client Requests

The client sends a command and its request ID to all replicas. On each replica, `DidaTradeMainServiceImpl`

1. applies the current debug mode,
2. stores the command in `RequestHistory`, and
3. tries to accept the request through `FastPaxosAcceptor`, and wakes `MainLoop` if it cannot accept it directly.

Paxos decides which request ID goes into each log slot. The command parameters are received directly from the client and kept locally for execution.

The server thread handling the client request waits until the command has been executed locally. It then replies with the result and execution instance, which the client uses for causal ordering. Balance requests take a separate path, described in Section 4.

3.2 Classic Multi-Paxos

In Classic Multi-Paxos, `MainLoop` schedules proposals only when this server is the current leader. It uses `PaxosLog` to assign pending requests to slots and `PaxosPipeline` to run

proposals concurrently. The path through our implementation is:

1. **Prepare the ballot.** The main loop calls `PaxosProposer.prepare()`. This runs phase one through `PeerRpc`, and the `PhaseOneResponseProcessor` collects promises and accepted entries from the acceptors. The proposer adopts the recovered values into `PaxosLog` and fills gaps with no-ops where no pending proposal is retained. A completed phase one is reused for later proposals in the same ballot.

2. **Propose a request ID.** For each slot, `MainLoop` asks `PaxosLog.assignPendingRequest()` for a `PaxosInstance`. This keeps an existing recovered value or assigns a pending request from `RequestHistory`. After claiming the slot, the main loop submits `PaxosProposer.propose()` to a pipeline worker. The proposer pins the value for that ballot and sends phase two through `PeerRpc`. On the receiving replica, `DidaTradePaxosServiceImpl.phasetwo()` obtains the instance and calls `DidaTradeServerState.processPhaseTwo()`. This checks membership and the current ballot, then records the acceptance in the instance. If accepted, `PaxosLearnerNotifier` sends Learn messages through `PeerRpc`, marked as classic votes.

3. **Record the decision.** Incoming Learn messages reach `DidaTradePaxosServiceImpl.learn()`, which passes each vote to `PaxosInstance.recordLearn()`. The instance counts distinct acceptors for each ballot and value. When a matching quorum decides the slot, the service wakes `MainLoop` so the executor can process it in log order. The leader also collects phase two replies through `PhaseTwoResponseProcessor`. Once those replies establish a quorum, `PaxosProposer` records the decision directly in the instance, without waiting for its own incoming Learns.

The classic quorum is two out of three acceptors in Schedule A and three out of five in Schedules B and C. Each slot's claim prevents two local workers from proposing it at the same time. When a proposal finishes, the worker wakes the main loop; failed attempts are retried after a short backoff, keeping the same phase two value for that slot and ballot. If an unsuccessful attempt reports a higher ballot, the replica updates its current ballot and stops proposing under the old one.

3.3 Fast Paxos

Fast Paxos lets acceptors vote directly for client requests without a leader proposal for each request. Schedule B has five acceptors and needs four matching votes, while classic recovery still uses a quorum of three.

The Fast path has three steps:

1. **Open the round.** The leader runs phase one and completes classic recovery of the earlier slots. It then sends `openfast`, specifying the first slot available for Fast requests. Earlier slots stay reserved for recovery. The leader tracks opening acknowledgements and retries until a Fast quorum acknowledges; opening on its own replica is not sufficient.

2. **Vote directly.** Each acceptor assigns incoming requests to local slots and sends Learn messages. Requests received before the opening are also considered. Different arrival orders can cause acceptors to assign different request IDs to the same slot.

3. **Count matching votes.** The learner stores votes in the instance's `LearnInfo`, separately by ballot and value. Each acceptor counts only once per ballot. Four matching votes decide the slot; four votes split between different values do not.

Handling collisions. A learner advances to the next ballot when either

- a Fast quorum has reported, but no value has a matching quorum, or
- the remaining acceptors cannot provide enough votes for any value to reach a Fast quorum.

For example, with five acceptors and a Fast quorum of four, three votes for different values already make a quorum impossible. Even if the two remaining acceptors vote for the same value, it can receive at most three votes.

The next leader runs phase one to recover accepted values and preserve any value that may already have been chosen. Automatic ballot changes use Learn evidence rather than timeouts. Since learners may receive different subsets of messages, the next leader may not detect the same collision. A ballot change through the console can still be needed in such cases.

3.4 Execution and Replies

Both paths end with decided slots in `PaxosLog`. The `OrderedPaxosExecutor` processes them in order:

1. **Check the next slot.** If it is not decided, execution waits. If it is decided but its command has not arrived from the client, the executor waits for that command instead of skipping the slot.

2. **Apply the command.** The executor checks `RequestHistory` before calling `TradeManager`. A request ID that has already been processed is not executed again, even if it appears in several slots. No-op

entries advance execution without changing the trading state.

3. **Store the result.** After execution, the result and execution instance are stored in the request record, and the request moves to the processed history. The waiting service handler is notified and replies to the client.

Applying the command and updating `last_executed_instance` happen under the same server-state lock used by balance reads, as explained in Section 4.

Execution may temporarily stop because a decided request ID arrives before its command. Registering the delayed client request wakes the main loop so execution can resume. This relies on the project's assumption that all replicas eventually receive every client request.

4 Causal Balance Reads

Each client tracks the largest execution instance from the replies it uses, initially -1, and includes it as `mininstance` in balance requests. The server waits until it has executed at least that slot, then returns its local balance and current execution instance. The client updates its known instance without decreasing it. Since commands are executed in log order, the returned balance reflects the client's earlier completed commands, even if a different replica answers the read. Balance requests do not go through Paxos.

The `DidaTradeServerState.getBalance()` method returns the balance and execution instance under the same server-state lock used by `OrderedPaxosExecutor` to apply commands and advance `last_executed_instance`. This keeps the balance and its token consistent.

5 Dynamic Membership

Schedule C changes the acceptor configuration at ballot 2. All seven replicas remain learners, but only five act as acceptors:

- **Before ballot 2:** replicas $\{0,1,2,3,4\}$.
- **From ballot 2:** replicas $\{2,3,4,5,6\}$.

A classic quorum requires three acceptors, and a Fast quorum requires four. Fast Paxos is used in ballots 0 and 2.

The new configuration must preserve decisions made by the old one. Although the two groups overlap, a quorum of the new group alone may not contain enough information to recover an old decision. The leader therefore completes recovery and transfers the earlier log before allowing new proposals in its ballot.

The handoff follows three steps:

1. **Recover the earlier slots.** The leader first runs phase one against its current acceptor group. If it cannot establish that the membership transfer has already been installed on a new acceptor, it also contacts the old group. The `PhaseOneResponseProcessor` then requires a recovery quorum from each group and selects values that preserve any earlier decision, including one that could have been chosen in a Fast ballot.

2. **Confirm the recovered prefix.** The leader reproposes the recovered slots through classic phase two in the current configuration. It waits until the recovered prefix is decided in its ballot before transferring it. New requests are not assigned during this handoff.

3. **Install and acknowledge.** The leader sends the recovered prefix to all learners. `StateTransfer` installs its decided IDs and, on current acceptors, also records them as accepted values. The `StateTransferResponseProcessor` counts acknowledgements only from the current acceptor group. Once three current acceptors acknowledge and the leader still leads that ballot, it records the ballot as completed.

Each replica records a `membership_transferred` marker when it installs a prefix from ballot 2 or later. Phase one reads the marker and accepted entries in the same locked snapshot. If a phase one recovery quorum of the new configuration includes a marked acceptor, its snapshot supplies the transferred prefix, so recovery can proceed without consulting the old group. A marker from a retired learner is not used for this purpose. The marker proves local installation, not that the transfer acknowledgement quorum has already been reached; the leader checks that separately.

6 AI Use and Authorship

AI-use declaration (Section 13).

All design choices and the code have been by us. The following tools, ChatGPT, Claude, Codex, and Cursor, have been used for explaining theoretical concepts and technical implementation details, assisting with code reviews and debugging, identifying potential issues, generating test cases, and supporting the drafting and review of the report. All AI-assisted contributions were reviewed and validated by the group.

Authorship declaration (Section 14).

We, João Agostinho (ist1109324), Martim Afonso (ist1106507), and Samuel Sacher (ist1120907), students of the Master's in Computer Science and Engineering (MEIC) at Instituto Superior Técnico, declare on our honor that the work entitled "DidaTrade: Replicated Trading Service," developed as part of the Distributed Applications Development course, resulted from the active participation of all members.

The individual contributions were:

- **Samuel Sacher:** Initial project setup, CI configuration and local tooling, implementation of Phase One in Multi Paxos (Step 2) and of causal ordering (Step 4), and some final refactoring work.

- **Martim Afonso:** Multi-Paxos design and implementation contributions (Step 2), extended regression and end-to-end testing, and Fast Paxos (Step 3).
- **João Agostinho:** Implementation of concurrent Paxos instances including proposal pipelining (Step 2), and dynamic membership (Step 5).

Step 1 was developed collaboratively, and all members participated in code reviews, testing, debugging, integration and refactoring on all steps.

We declare that the work is original and complies with the applicable academic regulations.

Lisbon, 16 October 2026

Signatures:
João Agostinho
Martim Afonso
Samuel Sacher